核桃编程

L12-3　模块1：模拟法入门
模块2：简单的模拟法

1. 单词卡

simulation

模拟

2. 知识探索

（1）模拟的概念

模拟法的本质就是要读懂题目含义，了解题目设定的规则，然后使用计算机语言，去模拟这些规则，最终得到结果的过程。

（2）走格子问题

从起始点向前一共有10个方格，编号分别是1,2,3...10。小队按照屏幕上的指令"前，前，后，前，前，后"，每次移动一个方格。假设0代表向前移动，1代表向后移动。求最终小队成员所在格子的编号。输入保证小队走过的格子都在1到10之间。

使用**res**变量表示小队所在的格子的编号。由于小队初始在编号为**1**的格子中，所以**res**变量初始化为**1**。

接着输入**n**个指令，我们使用**for**循环来模拟小队按照指令进行移动。循环变量**i**从**1**遍历到**n**，每一次循环表示执行一次指令。

在每一次循环中，输入指令，通过对指令的判断，决定小队是向前移动还是向后移动。如果是向前移动，那么**res**自增，如果向后移动那么**res**自减。最后**res**中存储的数值，就是小队最终所在的格子编号。

```
#include <iostream>
using namespace std;
int main()
{
    int res = 1;
    int n, t;
    cin >> n;
    for (int i = 1; i <= n; i++)
    {
        cin >> t;
        if (t == 0)
        {
            res++;
        }
        else
        {
            res--;
        }
    }
    cout << res;
    return 0;
}
```

（3）计算能量点问题

小队每次移动，要耗费1点能量。每消耗m点能量，小队会获赠1点能量。小队一开始有n点能量，最多可以移动多少次。

在编程中，由于不知道最终要循环多少次，可以使用**while**循环。

假设**num**表示移动的次数，**num**初始化为**0**。每次进入循环后，**n**自减，**num**自增。

定义变量**t**来模拟每消耗**3**点能量点就要增加**1**点。初始化**t**为**0**，每次进入**while**循环后，**t**自增，当**t**等于**3**时，需要对能量点**n**进行自增，同时需要将**t**赋值为**0**，便于下次统计。

```
int m, n;
cin >> m >> n;
// 使用num记录移动的次数
int num = 0;
// 使用t来计算小队是否可以获赠能量
// 当t等于m时可以获赠
// 获赠之后再将t赋值为0
int t = 0;
while (n > 0)
{
    n--;
    num++;
    t++;
    if (t == m)
    {
        n++;
        t = 0;
    }
}
cout << num;
return 0;
```

（4）发金币问题

城主给小核桃发送金币奖励，第1天发1枚，接下来的2天，每天发2枚，再接下来的3天，每天发3枚。以此类推。求第n天，发送的金币数。

使用嵌套循环来处理这个问题。外层循环没办法直接确定循环次数，我们使用**while(true)**。外层循环的循环变量是**t**，表示依次遍历金币数是**1,2,3**的情况。

内层循环我们使用**for**循环，循环变量**i**遍历**t**次，表示这**t**天每天需要送**t**每金币。由于每次进入内层循环，天数就自增**1**。因此我们设置一个变量**num**表示天数，**num**在**for**循环中自增。当天数等于**n**时，**t**就是当天赠送的金币数。我们直接将**t**输出，然后**return 0**。退出程序。

```
int n;
cin >> n;
// num记录天数
// 当num等于n时，停止模拟
int num = 0;
// t表示发送的金币数
// t从1累加
int t = 1;
while (true)
{
    for (int i = 1; i <= t; i++)
    {
        num++;
        if (num == n)
        {
            cout << t;
            return 0;
        }
    }
    t++;
}
```

（5）攻防大战问题

小队在密室中进行攻防大战，密室看作成一个平面，从左往右分割成n列，炮弹每次从第t列射出。而护盾的初始左右端点分别在第L列和第R列。左右移动护盾，每次只移动1列，要求阻挡住所有的炮弹攻击，求最少需要移动几次护盾。

假设炮弹在**t**列发出炮弹攻击。当**t**小于护盾的左边界**L**时，为了防御攻击，小队至少需要将护盾向左移动**L-t**次。移动之后，护盾的右边界**R**变为了**R-(L-t)**。

当**t**大于护盾的右边界**R**时，为了防御攻击，小队至少需要将护盾向右移动**t-R**次，移动之后，护盾的左边界**L**变为了**L+(t-R)**。

当 **L<=t<=R** 时，小队不需要移动护盾。

由于炮弹会攻击多次，我们使用**m**表示攻击的次数，使用**res**表示小队最少需要移动护盾的次数，**res**初始化为**0**。在**for**循环中，输入**t**，然后根据**t**的取值范围，分别进入不同的分支，模拟不同情况下小队移动护盾的情况。

```
int n, L, R;
cin >> n >> L >> R;
int m, t;
cin >> m;
// res表示移动的次数
int res = 0;
for (int i = 1; i <= m; i++)
{
    cin >> t;
    if (t < L)
    {
        int a = L - t;
        res += a;
        L = t;
        R = R - a;
    }
    else if (t > R)
    {
        int a = t - R;
        res += a;
        R = t;
        L = L + a;
    }
}
cout << res;
```